

这份实验报告是根据你提供的实验需求说明书编写的。报告涵盖了脚本设计思路、核心代码实现以及运行结果说明。

实验报告：Bash 脚本编程与日志分析

实验目的： 掌握 Bash 脚本的基本语法、文件处理、正则表达式以及常用的 Linux 文本处理工具（如 `awk`，`sort`，`uniq`）。

一、 基本题：文件扩展名统计

1. 实验要求

- 统计当前目录下所有文件的扩展名及其数量（不包括子目录）。
- 无扩展名的文件统计为 "no_ext"。
- 结果按扩展名字母顺序输出。

2. 设计思路

- 使用 `for file in *` 遍历当前目录，并利用 `[[ -f "$file" ]]` 过滤出普通文件。
- 通过参数扩展 `${file##*.}` 提取扩展名。
- 使用 **关联数组 (Associative Array)** 存储扩展名作为键 (Key)，数量作为值 (Value)。
- 利用 `sort` 对数组的键进行字母排序。

3. 代码实现

```
#!/bin/bash
declare -A counts
for file in *; do
    if [[ -f "$file" ]]; then
        if [[ "$file" != *.* ]]; then
            ext="no_ext"
        else
            ext="${file##*.}"
        fi
        ((counts["$ext"]++))
    fi
done

# 按字母顺序排序输出
for ext in $(echo "${!counts[@]}" | tr ' ' '\n' | sort); do
    echo "Extension: $ext, Count: ${counts[$ext]}"
done
```

二、 附加题：Nginx 日志分析器

1. 实验要求

-

基础统计：输出总请求数、唯一 IP 数和最常访问的 3 个资源路径。

-

状态码分析：计算各状态码比例，并用红色高亮显示 4xx 和 5xx 错误。

-

时间过滤：通过 `-s`（开始时间）和 `-e`（结束时间）参数过滤特定时段的日志。

2. 设计思路

-

参数处理：使用 `getopts` 处理命令行参数，并检查日志文件是否存在。

- **文本处理：**
- 使用 `awk` 解析 Nginx 日志格式。
- 通过字符串比对实现时间范围 (HH:MM:SS) 的逻辑过滤。
- **统计逻辑：**

-

`wc -l` 统计请求总数。

-

`awk '{print $1}' | sort -u` 统计唯一 IP。

-

`sort | uniq -c | sort -rn` 提取 Top 3 资源。

-

视觉反馈：使用 ANSI 转义序列 `\033[0;31m` 实现终端红色文本。

3. 核心代码实现 (access_log_analyzer.sh)

```
#!/bin/bash
RED='\033[0;31m'
NC='\033[0m'

LOG_FILE=$1
shift
while getopts "s:e:" opt; do
    case $opt in
        s) START="$OPTARG" ;;
        e) END="$OPTARG" ;;
    esac
done

# 过滤日志并存储在变量中
FILTERED=$(awk -v s="$START" -v e="$END" '
{
    split($4, t, ":");
    curr=sprintf("%s:%s:%s", t[2], t[3], t[4]);
    if (curr >= s && curr <= e) print $0;
}' "$LOG_FILE")

# 统计输出
echo "[Basic Statistics]"
echo "Total requests: $(echo "$FILTERED" | wc -l)"
echo "Unique IPs: $(echo "$FILTERED" | awk '{print $1}' | sort -u | wc -l)"
echo "Top 3 resources:"
echo "$FILTERED" | awk '{print $7}' | sort | uniq -c | sort -rn | head -n 3 | awk '{print NR". '

# 状态码高亮分析
echo -e "\n[Status Code Analysis]"
echo "$FILTERED" | awk -v r="$RED" -v n="$NC" '
{ code=$9; total++; if(code~/^4/) c4++; else if(code~/^5/) c5++; else other++; }
END {
    printf "4xx: %s%.1f%% (%d requests)%s\n", r, (c4/total)*100, c4, n;
    printf "5xx: %s%.1f%% (%d requests)%s\n", r, (c5/total)*100, c5, n;
}'
```

三、实验结果说明

1. 基础题输出示例

脚本将遍历目录，输出类似如下结果：

- Extension: csv, Count: 1
- Extension: md, Count: 2
- Extension: no_ext, Count: 0

2. 附加题输出示例

运行指令：`./access_log_analyzer.sh access.log -s 15:00:00 -e 16:30:00`

- 显示总请求数及唯一 IP 数。
- 列出排名前三的资源（如 `/api/data`）。
- 4xx 和 5xx 状态码将以红色显示在终端，便于快速定位错误。